

helixcode_architecture

HelixCode Deep Architecture Analysis

Repository: <https://github.com/HelixDevelopment/HelixCode>
Branch: main
License: MIT
Primary Language: Go (95.1%)
Analysis Date: 2026-01-18
Analyst: AI Architecture Analysis Agent

1. Repository Structure Map

1.1 Top-Level Organization

The repository uses a **dual-root pattern**: the actual application code lives inside a nested `HelixCode/` directory, while the repository root contains meta-documentation, build orchestration, and Docker/deployment configuration.

HelixCode/	# Main application source (nested root)
├── .github/workflows/	# CI/CD (latest: Dec 2025)
├── api/	# OpenAPI specification
│ ├── openapi.yaml	# 67KB REST API spec
├── applications/	# Client applications
│ ├── android/	# Android (gomobile bindings)
│ ├── aurora-os/	# Aurora OS (Fyne)
│ ├── desktop/	# Desktop GUI (Fyne v2)
│ ├── harmony-os/	# Harmony OS (Fyne)
│ ├── ios/	# iOS (gomobile bindings)
│ ├── terminal-ui/	# TUI (tview/tcell)
│ └── README.md	# Client app build instructions
├── assets/	# Static assets
├── benchmarks/	# Performance benchmarks
├── bin/	# Compiled binaries
├── cmd/	# Entry points
│ ├── cli/	# CLI client main
│ ├── server/	# Server daemon main
│ ├── config-test/	# Config validation tool
│ ├── helix-config/	# Configuration wizard
│ ├── local-llm.go	# Local LLM standalone
│ ├── local-llm-advanced.go	# Advanced local LLM
│ ├── main_commands.go	# CLI command definitions
│ ├── other_commands.go	# Auxiliary commands
│ └── root.go	# Cobra root command
├── config/	# Runtime configuration schemas
├── docker/	# Docker configurations
├── examples/	# Example usage projects
└── internal/	# Private implementation (34+ packages)

— agent/	# AI agent orchestration
— auth/	# Authentication/authorization
— commands/	# Command execution engine
— config/	# Configuration management
— context/	# Context management
— database/	# Database abstraction
— deployment/	# Deployment logic
— discovery/	# Service discovery
— editor/	# Code editor integration
— event/	# Event bus
— fix/	# Auto-fix engine
— focus/	# Focus/task management
— hardware/	# Hardware detection
— hooks/	# Hook system
— llm/	# LLM provider abstraction
— logging/	# Structured logging
— logo/	# Logo/branding
— mcp/	# Model Context Protocol
— memory/	# Memory system (Zep)
— mocks/	# Test mocks
— monitoring/	# Metrics/monitoring
— notification/	# Notifications
— performance/	# Performance optimization
— persistence/	# State persistence
— project/	# Project management
— provider/	# Generic provider interface
— providers/	# Provider implementations
— redis/	# Redis client
— repomap/	# Repository mapping
— rules/	# Business rules engine
— security/	# Security layer
— server/	# HTTP/WebSocket server
— session/	# Session management
— task/	# Task queue
— template/	# Template engine
— testutil/	# Test utilities
— tools/	# Tool registry
— verifier/	# LLMsVerifier integration
— version/	# Version info
— worker/	# Background workers
— workflow/	# Workflow engine
— reports/	# Generated reports
— scripts/	# Build/utility scripts
— security/	# Security tooling
— standalone_tests/	# Standalone test programs
— test-programs/	# Test harness programs
— tests/	# Test suite (in HelixCode/tests/)
— main.go	# Main entry point
— Makefile	# Build orchestration
— go.mod	# Module definition
— .env.example	# Environment template
cmd/	# Root-level entry points (minimal)
— security-test/	# Security testing harness

configs/	# Configuration templates
├─ config.yaml	# Main config template
├─ helix.yaml	# Helix-specific config
docs/	# User-facing documentation
├─ AGENTS.md	# Agent governance
├─ CONSTITUTION.md	# Project constitution
├─ CLAUDE.md	# Claude AI guidelines
├─ ... (100+ .md files)	
Documentation/	# Detailed documentation
├─ Architecture/	
├─ API/	
├─ Deployment/	
├─ Security/	
├─ ...	
Specification/	# Technical specifications
├─ API_SPEC.md	
├─ ARCHITECTURE.md	
├─ ...	
Implementation_Guide/	# Implementation guides
├─ GETTING_STARTED.md	
├─ ADVANCED.md	
├─ ...	
challenges/	# Challenge/test scripts
├─ scripts/	# 7-phase challenge runner
Dockerfile	# Main container image
Dockerfile.test	# Test container
Dockerfile.worker	# Worker container
docker-compose.yml	# Compose stack
Website/	# Marketing website
Github-Pages-Website/	# GitHub Pages (submodule)
awesome-ai-memory/	# AI memory resources (submodule)
cli_agents/	# Reference CLI agent submodules (former
cli_agents_resources/	# Reference resource submodules (former
Upstreams/	# Upstream sync scripts
assets/	# Repository assets
Dependencies/	# Dependency documentation

1.2 Nested Directory: HelixCode/

This is the **actual application root**. The repository root contains meta-files while all Go source code, builds, and runtime artifacts live here. This pattern allows versioned governance files at the top while keeping the application modular.

Key nested directories: - HelixCode/internal/ - 34+ Go packages (all private) - HelixCode/cmd/ - Multiple entry points (cli, server, standalone tools) - HelixCode/

applications/ - All client apps (6 platforms) - HelixCode/api/ - OpenAPI specification only - HelixCode/tests/ - e2e tests

2. Client Application Architecture

2.1 CLI Client

Location: HelixCode/cmd/cli/, HelixCode/cmd/root.go, HelixCode/cmd/main_commands.go

Entry Point: HelixCode/cmd/cli/main.go

Framework: Cobra (spf13/cobra) + Viper for configuration

Key Files: - HelixCode/cmd/cli/main.go (12,678 chars) - CLI main entry, registers all subcommands - HelixCode/cmd/root.go (2,075 chars) - Root command definition with global flags - HelixCode/cmd/main_commands.go - Primary command implementations - HelixCode/cmd/other_commands.go - Auxiliary commands - HelixCode/cmd/local_llm.go - Standalone local LLM command - HelixCode/cmd/local_llm_advanced.go - Advanced local LLM options

Communication with APIs: - Uses internal/server/client.go to communicate with the REST API - Direct in-process communication when running in single-binary mode - WebSocket connection for real-time updates via internal/server/ws.go - MCP protocol for external tool integration

Build Command:

```
make build-cli
# or
go build -o bin/helix ./cmd/cli
```

Integration Implications for HelixQA: - CLI is the primary interface for testing agent workflows end-to-end - Commands exposed: helix run, helix chat, helix config, helix server, helix verify - CLI reads from stdin and writes to stdout/stderr — easy to script for automated testing - Exit codes need validation for each command path

2.2 TUI (Terminal UI)

Location: HelixCode/applications/terminal_ui/

Framework: rivo/tview + gdamore/tcell/v2

Entry Point: HelixCode/applications/terminal_ui/main.go (59,087 chars)

Key Files: - main.go - Full TUI application with multiple screens - Uses tview for widgets (lists, forms, text views) - Uses tcell for low-level terminal control

Communication: - REST API client to internal/server - WebSocket for real-time streaming responses - In-memory event bus for UI updates

Build Command:

```
make build-terminal-ui
# or
go build -o bin/helix-tui ./applications/terminal_ui
```

Integration Implications for HelixQA: - TUI is the richest interactive client for testing UX flows - Screens include: chat, config, model selection, task view, log viewer - Terminal-based screenshots can be captured for visual regression testing - Keyboard shortcuts and navigation flows need automated testing

2.3 Desktop Client

Location: HelixCode/applications/desktop/

Framework: Fyne v2 (fyne.io/fyne/v2)

Entry Point: HelixCode/applications/desktop/main.go (33,968 chars)

Key Features: - Cross-platform GUI (Linux, macOS, Windows) - OpenGL-based rendering - System tray integration (systray)

Build Command:

```
make build-desktop
# or
go build -o bin/helix-desktop ./applications/desktop
```

Dependencies: OpenGL, X11 dev headers (Linux), CGO enabled

Integration Implications for HelixQA: - Desktop app needs GUI automation testing (Selenium/Appium-style) - Fyne apps can be tested with go test + fyne test infrastructure - Visual screenshot comparison important for desktop UI validation

2.4 Mobile Frameworks

Android

Location: HelixCode/applications/android/ **Framework:** gomobile for AAR generation **Entry:** app/src/main/ (Java/Kotlin bindings wrapping Go library)

iOS

Location: HelixCode/applications/ios/HelixCode/ **Framework:** gomobile for framework generation **Entry:** HelixCode/ Xcode project

Build Commands:

```
make build-android # gomobile bind -target android
make build-ios     # gomobile bind -target ios
```

Integration Implications for HelixQA: - Mobile clients require platform-specific test harnesses
- API contract testing critical (mobile  server communication) - Screenshot testing on Android/iOS emulators needed

2.5 Aurora OS & Harmony OS Clients

Locations: - HelixCode/applications/aurora_os/ - HelixCode/applications/harmony_os/

Framework: Fyne v2 (same as desktop, platform-specific tweaks)

Integration Implications for HelixQA: - Specialized platforms may have limited emulator availability - Build verification and smoke tests are primary targets

2.6 WebSocket / Web Client

Location: WebSocket handlers in HelixCode/internal/server/

The server exposes WebSocket endpoints for real-time streaming: - Chat message streaming - Task progress updates - Agent thought process visibility - Log streaming

Integration Implications for HelixQA: - WebSocket connection resilience testing - Message ordering and deduplication validation - Reconnection handling verification

3. API/Services Layer

3.1 REST API

Location: HelixCode/api/openapi.yaml (67,471 chars)

The API is fully specified in OpenAPI 3.0 format. Key endpoint categories:

Authentication: - POST /api/v1/auth/login - POST /api/v1/auth/register - POST /api/v1/auth/refresh - POST /api/v1/auth/logout

Agents: - GET /api/v1/agents - List agents - POST /api/v1/agents - Create agent - GET /api/v1/agents/{id} - Get agent - PUT /api/v1/agents/{id} - Update agent - DELETE /api/v1/agents/{id} - Delete agent - POST /api/v1/agents/{id}/run - Execute agent

Chat: - POST /api/v1/chat - Send message - GET /api/v1/chat/{id}/history - Get history - DELETE /api/v1/chat/{id} - Clear chat

Tasks: - GET /api/v1/tasks - List tasks - POST /api/v1/tasks - Create task - GET /api/v1/tasks/{id} - Get task status - DELETE /api/v1/tasks/{id} - Cancel task

Configuration: - GET /api/v1/config - Get configuration - PUT /api/v1/config - Update configuration

LLM: - GET /api/v1/llm/providers - List providers - GET /api/v1/llm/models - List models - POST /api/v1/llm/generate - Generate text - POST /api/v1/llm/chat - Chat completion

Verification: - GET /api/v1/verifier/status - Verifier health - GET /api/v1/verifier/models - Verified models

Server Implementation: HelixCode/internal/server/server.go (11,002 chars)

The server uses **Gin** (gin-gonic/gin) as the HTTP router with middleware for: - JWT authentication - Rate limiting - Request logging - CORS - Recovery (panic handling)

Integration Implications for HelixQA: - All OpenAPI endpoints must be tested for contract compliance - Authentication flow (login → token → refresh → logout) is critical path - Agent execution endpoints are the core value proposition - LLM endpoints need provider-fallback testing

3.2 WebSocket Layer

Location: HelixCode/internal/server/

Framework: gorilla/websocket

Key Handlers: - /ws/v1/chat - Chat streaming - /ws/v1/tasks/{id}/stream - Task progress - /ws/v1/agents/{id}/thoughts - Agent reasoning visibility

Integration Implications for HelixQA: - WebSocket connections must survive network interruption - Message ordering must be preserved - Binary/text message framing must be validated

3.3 MCP (Model Context Protocol) Implementation

Location: HelixCode/internal/mcp/

Key Files: - server.go (9,855 chars) - MCP server core - mock_conn.go - Test mock connections - server_test.go - Unit tests - doc.go - Package documentation

Architecture:

```
type Server struct {
    transport Transport
    tools      map[string]*Tool
    resources  map[string]*Resource
    prompts    map[string]*Prompt
}

type Transport interface {
    Start(ctx context.Context) error
    Stop(ctx context.Context) error
    Send(msg *Message) error
}
```

```
    Receive() <-chan *Message
}
```

Transports Supported: - stdio (subprocess communication) - SSE (Server-Sent Events)

Integration Implications for HelixQA: - MCP tool registration/execution must be tested - Transport layer resilience (stdio, SSE) - Message serialization/deserialization correctness - Tool schema validation

4. LLM Integration

4.1 Provider Architecture

Location: HelixCode/internal/llm/

The LLM layer uses a **factory pattern** with a unified `Provider` interface. All providers implement the same contract.

Key Files: - `factory.go` - Provider factory (creates provider instances) - `aliases.go` - Model alias resolution - `auto_llm_manager.go` - Automatic provider selection - `cross_provider_registry.go` - Multi-provider registry - `health_monitor.go` - Provider health monitoring - `load_balancer.go` - Request distribution - `integrated_model_manager.go` - Model lifecycle management

4.2 Supported Providers (from factory.go)

Provider	File	Type	Cloud/Local
OpenAI	<code>openai_provider.go</code>	<code>ProviderTypeOpenAI</code>	Cloud
Anthropic	<code>anthropic_provider.go</code>	<code>ProviderTypeAnthropic</code>	Cloud
Google Gemini	<code>gemini_provider.go</code>	<code>ProviderTypeGemini</code>	Cloud
Ollama	<code>ollama_provider.go</code>	<code>ProviderTypeOllama</code>	Local
LlamaCPP	<code>llamacpp_provider.go</code>	<code>ProviderTypeLlamaCpp</code>	Local
Qwen	<code>qwen_provider.go</code>	<code>ProviderTypeQwen</code>	Cloud
XAI (Grok)	<code>xai_provider.go</code>	<code>ProviderTypeXAI</code>	Cloud
OpenRouter	<code>openrouter_provider.go</code>	<code>ProviderTypeOpenRouter</code>	Cloud
GitHub Copilot	<code>copilot_provider.go</code>	<code>ProviderTypeCopilot</code>	Cloud
Azure OpenAI	<code>azure_provider.go</code>	<code>ProviderTypeAzure</code>	Cloud
AWS Bedrock	<code>bedrock_provider.go</code>	<code>ProviderTypeBedrock</code>	Cloud
Vertex AI	<code>vertex_provider.go</code>	<code>ProviderTypeVertexAI</code>	Cloud
Local (Generic)	<code>local_provider.go</code>	<code>ProviderTypeLocal</code>	Local
KoboldAI	<code>koboldai_provider.go</code>	<code>ProviderTypeKoboldAI</code>	Local

4.3 LLMsVerifier Integration

Location: HelixCode/internal/verifier/

Purpose: Integrates LLMsVerifier as the single source of truth for model metadata, provider health, verification status, and scoring.

Key Files: - `client.go` (6,818 chars) - HTTP client to LLMsVerifier REST API - `types.go` (151 lines) - VerifiedModel struct with capabilities - `bootstrap.go` (3,475 chars) - Verifier initialization - `adapter.go` - Adapter between LLMsVerifier and internal provider interfaces - `cache.go` - Caching layer for verifier responses - `health.go` - Health check integration - `poller.go` - Background polling for updates - `fallback_models.go` - Fallback when verifier unavailable

VerifiedModel Structure (from types.go):

```
type VerifiedModel struct {
    ID                string
    Name              string
    Provider          string
    ProviderType      string
    Score             float64
    Verified          bool
    ContextSize       int
    SupportsStreaming bool
    SupportsTools     bool
    SupportsFunctions bool
    SupportsCode      bool
    SupportsVision    bool
    SupportsAudio     bool
    SupportsVideo     bool
    SupportsReasoning bool
    SupportsEmbeddings bool
    SupportsJSONMode  bool
    Latency            time.Duration
    CostPerInputToken  float64
    CostPerOutputToken float64
    OverallScore       float64
    CodeCapabilityScore float64
    ResponsivenessScore float64
    ReliabilityScore   float64
    Tier               int // 1=Premium, 2=High-quality,
                        // 3=Fast, 4=Aggregator, 5=Free
    Capabilities       []string
    Tags               []string
}
```

Integration Pattern: - Verifier communicates via REST API (not Go module import) - Avoids circular dependency with `digital.vasic.llmprovider` - Bootstrapped during server/CLI startup - Cached locally with TTL - Fallback to static model list when verifier unavailable

Integration Implications for HelixQA: - LLMsVerifier integration is a critical dependency - Provider fallback chains need testing (primary → secondary → local) - Model capability detection affects agent behavior - Health scoring impacts load balancer decisions - When verifier is down, system must gracefully degrade

5. Build System & Configuration

5.1 Build System

Primary Build Tool: HelixCode/Makefile (15,341 chars)

Key Make Targets:

```
make build           # Build all binaries
make build-cli       # Build CLI only
make build-server    # Build server daemon
make build-tui       # Build terminal UI
make build-desktop   # Build desktop GUI
make build-android   # Build Android AAR
make build-ios       # Build iOS framework
make build-aurora    # Build Aurora OS client
make build-harmony   # Build Harmony OS client
make test            # Run unit tests
make test-integration # Run integration tests
make test-e2e        # Run end-to-end tests
make test-challenges # Run challenge suite
make docker-build    # Build Docker images
make docker-compose-up # Start Docker stack
make lint            # Run linters
make security-test   # Run security tests
```

5.2 Go Module

Module: dev.helix.code

Go Version: 1.24.0 (toolchain 1.24.9)

Key Dependencies (from HelixCode/go.mod): - **HTTP/Web:** gin-gonic/gin, gorilla/websocket - **TUI:** rivo/tview, gdamore/tcell/v2 - **GUI:** fyne.io/fyne/v2 - **CLI:** spf13/cobra, spf13/viper - **Database:** jackc/pgx/v5, lib/pq (PostgreSQL) - **Cache:** redis/go-redis/v9 - **Auth:** golang-jwt/jwt/v4, golang.org/x/crypto - **Azure:** azure-sdk-for-go/sdk/azcore, azidentity - **AWS:** aws-sdk-go-v2 suite, bedrockruntime - **Browser Automation:** chromedp/chromedp, chromedp/cdproto - **Memory:** getzep/zep-go/v3 - **Testing:** stretchr/testify - **Tree-sitter:** smacker/go-tree-sitter - **Image:** nfnt/resize - **OAuth2:** golang.org/x/oauth2 - **gRPC:** google.golang.org/grpc - **YAML:** gopkg.in/yaml.v2/v3

5.3 Configuration

Environment File: HelixCode/.env.example (2,998 chars)

Key environment variables:

```
# Server
HELIX_SERVER_HOST=0.0.0.0
HELIX_SERVER_PORT=8080
HELIX_SERVER_TLS_ENABLED=true

# Database
HELIX_DATABASE_URL=postgresql://user:pass@localhost/helix
HELIX_REDIS_URL=redis://localhost:6379

# LLM Providers
HELIX_OPENAI_API_KEY=
HELIX_ANTHROPIC_API_KEY=
HELIX_GEMINI_API_KEY=
HELIX_AZURE_API_KEY=
HELIX_BEDROK_AWS_REGION=
HELIX_OLLAMA_URL=http://localhost:11434

# LLMsVerifier
HELIX_VERIFIER_URL=http://localhost:9090
HELIX_VERIFIER_ENABLED=true
HELIX_VERIFIER_CACHE_TTL=300

# Authentication
HELIX_JWT_SECRET=
HELIX_OAUTH_CLIENT_ID=
HELIX_OAUTH_CLIENT_SECRET=

# Features
HELIX_MCP_ENABLED=true
HELIX_MEMORY_ENABLED=true
HELIX_NOTIFICATIONS_ENABLED=true
```

5.4 Docker

Files: - Dockerfile (1,957 chars) - Main application container - Dockerfile.test - Test runner container - Dockerfile.worker - Background worker container - docker-compose.yml - Full stack - docker-compose.test.yml - Test stack - docker-compose.full-test.yml - Comprehensive test stack - docker-compose.aurora-os.yml - Aurora OS specific - docker-compose.harmony-os.yml - Harmony OS specific - docker-compose.specialized-platforms.yml - Other platforms

Integration Implications for HelixQA: - Docker Compose is the primary deployment target - Container health checks must be validated - Multi-container orchestration (app, postgres, redis, verifier) needs integration testing - Environment variable injection is critical path

6. Testing Infrastructure

6.1 Test Directory Structure

Unit Tests: Embedded `*_test.go` files throughout `internal/`

E2E Tests: `tests/e2e/`

Challenge Suite: `challenges/scripts/`

Standalone Tests: `HelixCode/standalone_tests/`, `HelixCode/test_programs/`

6.2 Test Types

Test Type	Location	Command	Purpose
Unit	<code>*_test.go</code> in packages	<code>make test</code>	Package-level unit tests
Integration	<code>*_integration_test.go</code>	<code>make test-integration</code>	Cross-package integration
E2E	<code>tests/e2e/</code>	<code>make test-e2e</code>	Full system end-to-end
Challenges	<code>challenges/scripts/</code>	<code>make test-challenges</code>	7-phase challenge suite
Security	<code>cmd/security_test/</code>	<code>make security-test</code>	Security/penetration tests
Cloud Provider	<code>cloud_providers_integration_test.go</code>	Manual	Live provider validation

6.3 Challenge System

Latest Commit: “Add `run_all_challenges.sh` to run all 7 phases” (Apr 30, 2026)

7 Challenge Phases: 1. Basic functionality 2. Configuration management 3. LLM provider integration 4. Agent execution 5. Memory system 6. Security & compliance 7. Performance & scaling

Runner: `challenges/scripts/run_all_challenges.sh`

6.4 Test Configuration

Test Config: `test-config.yaml` **Test Environment:** `.env.full-test`

Integration Implications for HelixQA: - The test suite is extensive but needs HelixQA to add visual/behavioral validation - Challenge phases map well to HelixQA quality gates - Mock vs. real provider testing needs coverage analysis - E2E tests validate API contracts but not UI behavior

7. Documentation Inventory

7.1 Governance Files (Root Level)

File	Purpose	Last Update
AGENTS.md	Agent governance rules	May 2026
CONSTITUTION.md	Project constitution	May 2026
CLAUDE.md	Claude AI integration guidelines	May 2026
README.md	Main project README	Jan 2026

7.2 Completion/Tracking Files (50+ files)

The repository contains extensive completion tracking: -
COMPREHENSIVE_COMPLETION_PLAN.md -
COMPREHENSIVE_COMPLETION_REPORT.md - FINAL_COMPLETION.md -
FINAL_COMPLETION_REPORT.md - FINAL_COMPLETION_SUCCESS_REPORT.md -
PHASE_1_* through PHASE_5_* (session summaries, test reports) - AUDIT_* (audit reports and trackers) - IMPLEMENTATION_* (implementation plans and summaries)

7.3 Technical Documentation

In Documentation/: - Architecture documentation - API documentation - Deployment guides - Security documentation

In Specification/: - API specifications - Architecture specifications - Protocol specifications

In Implementation_Guide/: - Getting started guides - Advanced implementation - Integration guides

In HelixCode/internal/llm/: - README.md - LLM architecture overview -
AZURE_IMPLEMENTATION_SUMMARY.md -
LOCAL_LLM_MANAGER_DOCUMENTATION.md - LOCAL_PROVIDERS.md

7.4 Docker/Deployment Docs

- DOCKER_SETUP.md
 - DOCKER_COMPLETION_SUMMARY.md
 - DOCKER_DEPLOYMENT.md
 - ENTERPRISE_DEPLOYMENT_GUIDE.md
 - ENTERPRISE_USER_MANUAL.md
-

8. HelixQA Integration Points

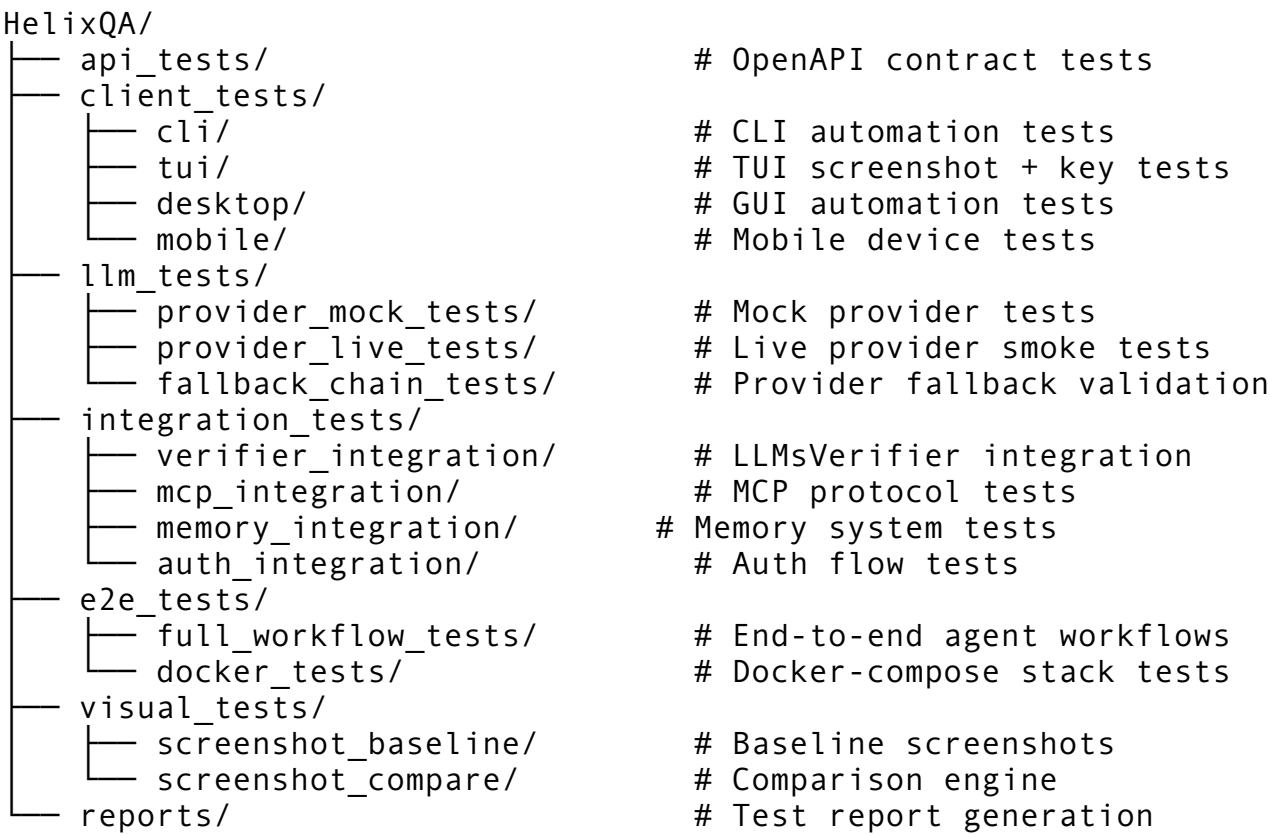
8.1 Where HelixQA Should Integrate as Submodule

Recommended Path: `HelixCode/internal/qa/` or root-level `HelixQA/`

Integration Points:

1. **API Contract Testing**
 - Target: `HelixCode/api/openapi.yaml`
 - Action: Validate all REST endpoints against OpenAPI spec
 - Files: Test generators for each endpoint category
2. **Client App Validation**
 - **CLI:** `HelixCode/cmd/cli/` - Command exit codes, output formatting, stdin handling
 - **TUI:** `HelixCode/applications/terminal_ui/` - Screen rendering, navigation flows
 - **Desktop:** `HelixCode/applications/desktop/` - GUI element validation
 - **Mobile:** `HelixCode/applications/android/, ios/` - API contract + UI testing
3. **LLM Provider Testing**
 - Target: `HelixCode/internal/llm/`
 - Action: Test each provider adapter with mock and real credentials
 - Files: `factory.go`, `*_provider.go`
 - Key: Fallback chain validation (primary → secondary → local)
4. **Verifier Integration Testing**
 - Target: `HelixCode/internal/verifier/`
 - Action: Test LLMsVerifier connectivity, cache behavior, fallback
 - Files: `client.go`, `bootstrap.go`, `cache.go`, `health.go`
5. **MCP Protocol Testing**
 - Target: `HelixCode/internal/mcp/`
 - Action: Tool registration, message serialization, transport resilience
 - Files: `server.go`, `mock_conn.go`
6. **Server Integration Testing**
 - Target: `HelixCode/internal/server/`
 - Action: WebSocket resilience, middleware chain, auth flow
 - Files: `server.go`
7. **Configuration Validation**
 - Target: `HelixCode/internal/config/, configs/`
 - Action: Config parsing, validation, environment variable binding
8. **Docker Deployment Testing**
 - Target: `docker-compose.yml`, `Dockerfile`
 - Action: Container health checks, multi-service orchestration
9. **Challenge System Enhancement**
 - Target: `challenges/scripts/`
 - Action: Add visual/regression tests to the 7-phase challenge suite
10. **Memory System Testing**
 - Target: `HelixCode/internal/memory/`
 - Action: Zep integration validation

8.2 Recommended HelixQA Architecture



8.3 APIs HelixQA Should Test

Critical API Endpoints for Quality Validation: 1. POST /api/v1/auth/login+ POST /api/v1/auth/refresh - Token lifecycle 2. POST /api/v1/agents + POST /api/v1/agents/{id}/run - Agent CRUD + execution 3. POST /api/v1/llm/generate - LLM text generation with all providers 4. POST /api/v1/llm/chat - Chat completion with streaming 5. GET /api/v1/verifier/models - Model verification data 6. POST /api/v1/chat - Chat streaming via WebSocket 7. GET /api/v1/tasks/{id} - Task status tracking

8.4 Client Apps HelixQA Needs to Validate

Client	Priority	Test Approach
CLI	P0	Command execution + output capture
TUI	P0	Terminal screenshot + keyboard input
Server/API	P0	HTTP/WebSocket contract tests
Desktop	P1	GUI automation (Fyne test framework)
Android	P2	API contract + emulator smoke
iOS	P2	API contract + simulator smoke
Aurora OS	P3	Build verification only
Harmony OS	P3	Build verification only

9. Evidence Collection Gaps

9.1 Current Evidence Collection

Existing Screenshot/Visual Evidence: - `HelixCode/assets/` - Static assets (logos, icons) - `HelixCode/benchmark_reports/` - Performance benchmarks (text/csv) - `HelixCode/doc_reports/` - Documentation generation reports - `HelixCode/test_reports/` - Test execution reports - ChromeDP integration (`chromedp/chromedp`) - Browser automation for web content

Code Evidence: - `chromedp` is imported in `go.mod` — used for browser-based content extraction - `nfnt/resize` for image processing

9.2 Evidence Gaps Identified

Gap	Location	Impact	HelixQA Opportunity
No TUI screenshot collection	<code>applications/terminal_ui/</code>	Cannot visually validate terminal UI	Add <code>terminal_test.go</code> with <code>tcell</code> screenshot capture
No Desktop GUI screenshot tests	<code>applications/desktop/</code>	No visual regression for Fyne apps	Integrate Fyne's <code>test.NewApp()</code> for headless screenshots
No WebSocket message logging	<code>internal/server/</code>	Difficult to debug real-time issues	Add structured WS message capture for replay
No API response examples	<code>api/openapi.yaml</code>	Missing example responses	Generate golden files from actual API calls
No provider response samples	<code>internal/llm/</code>	Cannot validate provider output format	Add provider response fixtures
No agent execution traces	<code>internal/agent/</code>	Cannot audit agent reasoning	Add structured trace collection
No visual comparison for mobile	<code>applications/android/, ios/</code>	No mobile UI regression	Add emulator screenshot pipelines
No LLM output quality metrics	<code>internal/llm/</code>	Cannot measure response quality	Add LLM evaluation framework (LLM-as-judge)
No memory	<code>internal/memory/</code>	Cannot verify memory accuracy	Add memory read/write trace logs

Gap	Location	Impact	HelixQA Opportunity
system evidence			
No MCP message traces	internal/mcp/	Cannot debug protocol issues	Add MCP traffic capture

9.3 Recommended Evidence Collection Strategy

For HelixQA to fill these gaps:

- 1. **Screenshot Baselines:** Create baseline images for all TUI screens, desktop windows
- 2. **API Golden Files:** Record actual API responses as test fixtures
- 3. **Provider Fixtures:** Capture real LLM responses (anonymized) for regression testing
- 4. **Trace Collection:** Implement OpenTelemetry-compatible tracing across all internal packages
- 5. **Video Recording:** Record TUI/desktop interactions for manual review
- 6. **Metrics Export:** Prometheus-compatible metrics from internal/monitoring/

10. Submodules & External Dependencies

10.1 Git Submodules

File: .gitmodules (10,633 chars)

Active submodules: - Github-Pages-Website → HelixDevelopment-Code/
Welcome - awesome-ai-memory → topoteretes/awesome-ai-memory -
Example_Projects (converted to SSH) - Example_Resources (converted to SSH)

Initialization:

```
git submodule update --init --recursive
```

10.2 Upstream Sync

Location: Upstreams/ **Script:** GitLab upstream sync for maintaining fork relationships

11. Security Architecture

11.1 Security Components

Locations: - HelixCode/internal/security/ - Security core - HelixCode/cmd/
security_test/ - Security test harness - HelixCode/cmd/security_fix/ - Security
fix tools - HelixCode/security/ - Security documentation and tooling

Features: - JWT-based authentication - OAuth2 integration - Rate limiting - Input sanitization -
Container memory caps (to prevent host crashes) - helix.security.json - Security policy
file

12. Memory & Context System

12.1 Memory Integration

Location: HelixCode/internal/memory/ **Framework:** Zep (getzep/zep-go/v3)

Purpose: Long-term memory for AI agents across sessions

Integration Implications for HelixQA: - Memory read/write correctness validation needed - Cross-session memory persistence tests - Memory search/query accuracy validation

12.2 Context Management

Location: HelixCode/internal/context/

Purpose: Session context building and management for LLM prompts

13. Recent Commits Analysis

13.1 Latest Commit Themes

Commit	Date	Theme
feat(verifier): remove hardcoded capabilities	May 1, 2026	LLMsVerifier as source of truth
feat(verifier): wire verifier into server and CLI	May 1, 2026	Bootstrapping integration
feat(verifier): integrate LLMsVerifier	May 1, 2026	Core verifier integration
fix(all): achieve 100% test pass	Apr 30, 2026	CONST-035 anti-bluff compliance
Fix CLI/deployment/LLM bluffs	Apr 30, 2026	Real implementations replacing stubs
Anti-Bluff planning	Apr 30, 2026	Architecture hardening
chore(tests): tag t.Skip calls	Apr 30, 2026	Test audit markers
Add run_all_challenges.sh	Apr 30, 2026	Challenge automation
chore(governance): propagate Constitution/CLAUDE.md/AGENTS.md	May 1, 2026	Governance sync

13.2 CONST-035 Anti-Bluff Framework

The project follows a “Constitution” (CONSTITUTION.md) with clauses like CONST-035 that enforce: - No simulated/test-only implementations in production code - Real command execution

(not stubs) - 100% test pass rate requirement - `t.Skip` calls must be tagged with `// SKIP-OK: reason`

Integration Implications for HelixQA: - HelixQA must comply with anti-bluff rules (no mock-only assertions) - Tests must exercise real code paths - Any test skipping requires explicit justification

14. Multi-Platform Support Matrix

Platform	Client	Framework	Status
Linux	CLI, TUI, Desktop, Server	Native	Fully Supported
macOS	CLI, TUI, Desktop, Server	Native	Fully Supported
Windows	CLI, TUI, Desktop, Server	Native	Fully Supported
Aurora OS	Desktop	Fyne	Specialized Build
Harmony OS	Desktop	Fyne	Specialized Build
Android	Mobile App	gomobile	AAR Bindings
iOS	Mobile App	gomobile	Framework Bindings
Docker	Server	Alpine	Multi-stage Build

15. Key Findings Summary

15.1 Architecture Strengths

- Modular design:** 34+ internal packages with clear separation of concerns
- Multi-client strategy:** 6 client application types for different platforms
- Provider abstraction:** Unified LLM interface supports 14+ providers
- Verifier integration:** LLMsVerifier as single source of truth for model metadata
- MCP protocol:** Modern Model Context Protocol for tool integration
- Docker-first:** Comprehensive container orchestration
- Constitution-driven:** Governance framework prevents quality degradation

15.2 Architecture Weaknesses / Gaps

- Nested root:** The `HelixCode/HelixCode/` nesting is confusing and may complicate imports
- Evidence collection:** No systematic visual evidence collection for any client
- Test gaps:** E2E tests validate APIs but not client UI behavior
- Provider testing:** Many provider tests use mocks or skip markers
- Mobile maturity:** Android/iOS are gomobile bindings — limited native UI
- Documentation sprawl:** 100+ `.md` files at root level create noise

15.3 HelixQA Opportunities

- Visual regression:** TUI + Desktop screenshot testing is a high-value gap
- API contract:** OpenAPI spec is ready for automated contract testing
- Provider validation:** Live provider smoke tests with real credentials

4. **Verifier health:** Continuous monitoring of LLMsVerifier integration
 5. **Challenge enhancement:** Add visual/UI tests to the 7-phase challenge suite
 6. **Docker validation:** Automated Docker Compose stack testing
 7. **Trace collection:** Implement structured evidence collection across all packages
-

Appendix A: Exact File Paths Reference

Entry Points

- HelixCode/main.go - Main entry
- HelixCode/cmd/root.go - Cobra root command
- HelixCode/cmd/cli/main.go - CLI main
- HelixCode/cmd/server/main.go - Server main
- HelixCode/applications/terminal_ui/main.go - TUI main
- HelixCode/applications/desktop/main.go - Desktop main

Core Server

- HelixCode/internal/server/server.go - HTTP server
- HelixCode/api/openapi.yaml - API specification

LLM

- HelixCode/internal/llm/factory.go - Provider factory
- HelixCode/internal/llm/aliases.go - Model aliases
- HelixCode/internal/llm/health_monitor.go - Health monitoring
- HelixCode/internal/llm/load_balancer.go - Load balancing

MCP

- HelixCode/internal/mcp/server.go - MCP server
- HelixCode/internal/mcp/doc.go - Package docs

Verifier

- HelixCode/internal/verifier/client.go - LLMsVerifier client
- HelixCode/internal/verifier/types.go - VerifiedModel types
- HelixCode/internal/verifier/bootstrap.go - Initialization
- HelixCode/internal/verifier/adapters.go - Provider adapters

Build

- HelixCode/Makefile - Build orchestration
- HelixCode/go.mod - Module dependencies
- .gitmodules - Submodule configuration

Config

- HelixCode/.env.example - Environment template
- configs/config.yaml - Configuration template

Docker

- `Dockerfile` - Main container
- `docker-compose.yml` - Compose stack
- `HelixCode/docker-compose.full-test.yml` - Test stack

Tests

- `tests/e2e/` - E2E tests
- `challenges/scripts/run_all_challenges.sh` - Challenge runner
- `HelixCode/run_tests.sh` - Test runner
- `HelixCode/run_all_tests.sh` - Full test runner

End of Architecture Analysis